Project

In this project, you will design and implement your own relational database from scratch. The goal is to apply the concepts we have covered to a topic of your choosing. You will be responsible for identifying a realistic use case, defining appropriate relations, and constructing a well-structured schema. By the end of the project, your database should demonstrate thoughtful design, proper use of constraints, and the ability to query meaningful data effectively.

Requirements:

1. At least 5 relations.
2. Each relation should have at least 3 attributes.
3. If you choose to add a relation that has less than 3 attributes, it must be in addition to the minimum 5.
4. Each relation should have a primary key.
5. At least 2 relations should have at least 1 foreign key.
6. Each relation should have at least one constraint.
7. All relations should have at least 6 records. The records must have realist information. However, it does not have to be factual.
8. At least 1 role with the password 'Project@520'. The role can have any privilege(s) you choose.
9. At least 1 view that can be accessed by the created role.
10. Your database cannot be of a University, Bank, Crime Scene Report, or Employee.

Task 1 (1 point):

Open your GitHub repository in Codespace.

1. In the terminal run the command: **`pip install -r requirements.txt`**
2. In the terminal run the command: **`cd Project`**
3. In the terminal run the command: **`cp .env.example .env`**
4. In the terminal run the command: **`mkdir DATA`**
5. In the terminal run the command: **`touch ddl.sql`**
6. In the **.env** file, paste the copied connection string from task 1 after: **`DB_CONNECTION_STRING=`**
7. You'll now need to parse the connection string to get the **`DB_HOST`** and **`DB_PASSWORD`** values. Example:

```
postgresql://neondb_owner:password@ep-example-123456-pooler.us-east-1.aws.neon.tech/neondb?sslmode=require&channel_binding=require
```

8. The **`host`** can be found between **`@`** and the first **`/`** after **`@`**. The **`password`** can be found between the first **`:`** before **`@`** and **`@`** in the connection string.

```
DB_HOST=ep-example-123456-pooler.us-east-1.aws.neon.tech
DB_PASSWORD=password
```

Task 2 (2 points):

Inside the **ddl.sql** file, list all necessary CREATE TABLE IF NOT EXISTS statements.

Task 3 (2 points):

Inside the DATA/ folder create all necessary CSV files. The name of each file should equal the name of its corresponding relation. The first line of each file should have the names of the attributes for the relation separated by a comma (no spaces).

Task 4 (1 point):

Create a file called **Database.py** inside **Project/**. At the top of the file, import ONLY the following:

```
psycopg2
sql from psycopg2
os
tabulate from tabulate
load_dotenv from dotenv
```

Create a class named **Database** that is identical to the Database class from Homework10. Modify the following two methods.

1. `def __init__(self, db_name=None, ddl_file="ddl.sql", data_dir="DATA") -> None:`
   Define the following variables:

   self.connection assigned `None`
   self.ddl_file assigned `ddl_file`
   self.data_dir assigned `data_dir`
   self.relations assigned `[]`
   self.db_name assigned `db_name`
2. `def connect(self, username: str = None, password: str = None) -> None:`
   a. Connect to the database using the **connect_to_db()** function from setup.py as a guide. Instead of returning the connection/None, assign it to self.connection.
   b. For the **database** variable assignment, provide a ternary expression that assigns `os.getenv("DB_NAME")` to **database** if **self.db_name** is None. Otherwise it assigns **self.db_name** to **database**. (This should be a one-liner).
   c. For the **user** variable assignment, provide a ternary expression that assigns `os.getenv("DB_USER")` to **user** if `username` is None. Otherwise it assigns `username` to **user**. (This should be a one-liner).
   d. For the **password** variable assignment, provide a ternary expression that assigns `os.getenv("DB_PASSWORD")` to **password** if `password` is None. Otherwise it assigns `password` to **password**. (This should be a one-liner.)

Task 5 (2 points):

Create a main.cpp file that handles the following:

1. Import your Database class from Database.py.
2. Call the create_database() static function of the Database class with the name of your database as its argument.
3. Create as Database object with db_name="your_database" as the method argument. (Replace "your_database" with the name of your database.)
4. Connect to your database.
5. Setup your database.
6. Create the required roles and views.
7. Execute the following queries:
   a. A select statement to get some rows from one of the tables
   b. A select statement on two or more joined tables.
   c. A update statement.
   d. A delete statement.
   e. A statement for at least 2 aggregate functions.
8. Disconnect from the database.
9. Establish another connection to your database as the created role.
10. Execute the following queries:
    a. A select statement on the associated view,
11. Disconnect from the database.

Task 6 (2 points):
Create a schema diagram for your database and upload it to the Project/ folder. It can be hand drawn or digitally constructed.

Push your work to GitHub using the following commands:

```
git pull
git add .
git –m “Project upload”
git push
```